

Dokumentasi Project

Backend Wisata Puncak Mas

Backend API — NestJS



Dibuat Oleh:

Muhammad Tammam

NPM: 23312202

Kelas: IF 23 E

Universitas Teknokrat Indonesia

1. Gambaran Umum Project

Project ini adalah sebuah sistem Backend yang digunakan untuk mengelola data pengunjung dan transaksi di wisata Puncak Mas.

Bagian	Teknologi	Tugasnya Apa?
Backend (Server API)	NestJS + TypeORM + MySQL	Menerima permintaan dari frontend, mengolah data, lalu menyimpan ke database

Dokumentasi ini secara khusus membahas bagian Backend API yang dibangun menggunakan NestJS. Server backend berjalan di alamat `http://localhost:3000` dan melayani semua permintaan data dari frontend.

2. Teknologi yang Digunakan

Berikut adalah daftar teknologi dan library yang dipakai pada bagian backend:

Teknologi	Kegunaan
NestJS	Framework utama untuk membangun server backend berbasis TypeScript. Mengatur struktur kode menjadi Controller, Service, Module, dan Guard.
TypeORM	Jembatan antara kode TypeScript dengan database MySQL. Mempermudah operasi baca dan tulis data tanpa harus menulis query SQL secara manual.
MySQL (XAMPP)	Tempat penyimpanan data pengunjung dan transaksi wisata secara permanen.
class-validator	Library untuk memvalidasi data yang masuk. Memastikan format data dari frontend sudah benar sebelum diolah.
TypeScript	Bahasa pemrograman yang digunakan di seluruh kode backend. Mencegah bug akibat tipe data yang salah.

3. Struktur File Backend (folder src/)

Semua kode backend berada di dalam folder `src/` dengan rincian file sebagai berikut:

File / Folder	Fungsi Singkat
<code>src/main.ts</code>	Titik awal aplikasi berjalan. Mengatur server, izin akses (CORS), dan validasi data global.
<code>src/app.module.ts</code>	Pusat pendaftaran aplikasi. Menghubungkan ke database dan mendaftarkan semua modul fitur.
<code>src/app.controller.ts</code>	Menangani request ke alamat utama <code>"/</code> dan mengembalikan teks <code>"Hello World!"</code> .

File / Folder	Fungsi Singkat
src/app.service.ts	Berisi logika sederhana untuk fungsi getHello().
src/wisata/wisata.module.ts	Menyatukan semua komponen fitur wisata (controller, service, entity) menjadi satu paket.
src/wisata/wisata.controller.ts	Mengatur semua alamat (endpoint) API untuk data wisata, seperti /wisata dan /wisata/:id.
src/wisata/wisata.service.ts	Tempat logika utama seperti menghitung total harga, menyimpan data, dan mengambil data dari database.
src/wisata/wisata.guard.ts	Sistem keamanan. Memeriksa apakah request yang masuk membawa kunci akses yang benar sebelum data boleh diubah.
src/wisata/entities/wisata.entity.ts	Gambaran struktur tabel "wisata" di database. TypeORM membaca file ini untuk mengetahui kolom apa saja yang ada.
src/wisata/dto/create-wisata.dto.ts	Menentukan data apa saja yang wajib dikirim saat membuat data baru, beserta aturan validasinya.
src/wisata/dto/update-wisata-dto.ts	Sama seperti DTO Create, tetapi semua field bersifat opsional karena hanya memperbarui sebagian data.
src/wisata/*.spec.ts	File-file pengujian otomatis (unit test) untuk memastikan masing-masing komponen berfungsi dengan benar.

4. Titik Awal Server (main.ts)

File main.ts adalah pintu masuk utama aplikasi. Saat perintah "npm run start" dijalankan, file inilah yang pertama kali dieksekusi. Di dalamnya ada fungsi bootstrap() yang bertugas menghidupkan server.

Tiga hal utama yang dikonfigurasi di sini:

- **Izin Akses (CORS):** Mengizinkan frontend Next.js (yang berjalan di port berbeda) untuk bisa berkomunikasi dengan server backend ini. Tanpa ini, browser akan memblokir koneksi.
- **Validasi Global:** Memasang "penjaga" yang secara otomatis memeriksa format data di setiap request yang masuk. Jika data tidak sesuai aturan DTO, request akan langsung ditolak sebelum masuk ke proses lebih lanjut.
- **Port Server:** Server berjalan di port 3000 secara default. Port bisa diubah melalui variabel lingkungan (environment variable) PORT.

Detail Konfigurasi Validasi

Pengaturan	Nilai	Artinya
whitelist	true	Data yang tidak ada di DTO akan langsung dibuang secara otomatis, tidak akan diteruskan ke proses berikutnya.

Pengaturan	Nilai	Artinya
transform	true	Data yang masuk akan diubah otomatis ke tipe yang sesuai. Contoh: teks "1" akan diubah menjadi angka 1.
enableImplicitConversion	true	Konversi tipe data berlaku otomatis tanpa perlu menulis dekorator tambahan di setiap field.

5. Pengaturan Database (app.module.ts)

File app.module.ts adalah pusat pengaturan aplikasi. Di sinilah koneksi ke database MySQL dikonfigurasi menggunakan TypeORM.

Sebelum menjalankan backend, pastikan database sudah dibuat terlebih dahulu di phpMyAdmin dengan nama puncak_mas_db.

Pengaturan	Nilai	Keterangan
type	mysql	Jenis database yang digunakan adalah MySQL.
host	localhost	Database berjalan di komputer yang sama (lokal).
port	3306	Port standar untuk MySQL.
username	root	Nama pengguna bawaan dari XAMPP.
password	(kosong)	Password default XAMPP tidak menggunakan kata sandi.
database	puncak_mas_db	Nama database yang harus sudah dibuat di phpMyAdmin.
synchronize	true	Jika struktur entity diubah, tabel di database akan ikut berubah secara otomatis. Fitur ini hanya untuk tahap pengembangan (development).

6. Struktur Data di Database (wisata.entity.ts)

File entity adalah gambaran langsung dari tabel yang ada di database. TypeORM membaca file ini untuk mengetahui kolom apa saja yang perlu dibuat di tabel "wisata".

Setiap properti di file entity merepresentasikan satu kolom di tabel database:

Kolom di Database	Nama di Kode	Tipe Data	Keterangan
id	id	INT	Nomor unik yang diisi otomatis oleh database (auto increment). Berfungsi sebagai identitas setiap baris data.
nama	nama	VARCHAR	Nama pengunjung yang melakukan transaksi.

Kolom di Database	Nama di Kode	Tipe Data	Keterangan
tanggal	tanggal	DATETIME	Tanggal dan waktu kunjungan. Jika tidak diisi, otomatis menggunakan waktu saat data dibuat.
jumlah	jumlah	INT	Jumlah tiket atau jumlah orang yang berkunjung.
harga_tiket	hargaTiket	INT	Harga per tiket. Nama kolom di database adalah harga_tiket, tetapi di kode ditulis sebagai hargaTiket mengikuti konvensi camelCase.
total	total	INT	Total biaya yang harus dibayar. Nilainya dihitung otomatis oleh service (jumlah x harga), bukan diinput manual.

7. Aturan Input Data (DTO)

DTO (Data Transfer Object) adalah aturan yang menentukan data apa saja yang harus dikirim oleh frontend, dan bagaimana format datanya. Jika data tidak sesuai aturan, server akan langsung menolak request tersebut dan mengirimkan pesan kesalahan.

7.1 DTO untuk Menambah Data Baru (CreateWisataDto)

Digunakan saat frontend mengirim request POST /wisata. Semua field di bawah ini wajib diisi.

Field	Tipe Data	Aturan Validasi	Keterangan
Tanggal	string	Wajib. Format tanggal ISO.	Contoh format yang benar: "2024-08-17". Format lain akan ditolak.
Nama	string	Wajib. Minimal 3, maksimal 100 karakter.	Nama pengunjung. Tidak boleh kosong.
Jumlah	number	Wajib. Nilai minimal 1.	Jumlah tiket yang dibeli. Tidak boleh 0 atau negatif.
Harga	number	Wajib. Nilai minimal 0.	Harga per tiket. Boleh 0 (misalnya untuk tiket gratis).

7.2 DTO untuk Mengubah Data (UpdateWisataDto)

Digunakan saat frontend mengirim request PATCH /wisata/:id. Semua field bersifat opsional, artinya frontend hanya perlu mengirim field yang ingin diubah, tidak harus semuanya.

Field	Tipe Data	Status	Keterangan
Tanggal	string	Opsional	Kirim hanya jika ingin mengubah tanggal kunjungan.
Nama	string	Opsional	Kirim hanya jika ingin mengubah nama pengunjung.
Jumlah	number	Opsional	Kirim hanya jika ingin mengubah jumlah tiket.
Harga	number	Opsional	Kirim hanya jika ingin mengubah harga tiket.

8. Sistem Keamanan API (wisata.guard.ts)

WisataGuard adalah sistem keamanan yang bekerja seperti satpam. Setiap kali ada request yang ingin menambah, mengubah, atau menghapus data, guard ini akan memeriksa apakah request tersebut membawa kunci akses yang benar.

Cara Kerja Guard

- Guard memeriksa bagian **Authorization** pada header request yang masuk.
- Jika header berisi kunci yang benar (**Admin123** atau **Bearer Admin123**), request diizinkan untuk dilanjutkan.
- Jika kunci salah atau tidak ada sama sekali, server menolak request dan mengirimkan pesan error: **"Akses Ditolak: Kunci CMS tidak valid!"**

Contoh header yang benar saat mengirim request:

```
Authorization: Admin123
```

atau

```
Authorization: Bearer Admin123
```

Endpoint yang Dilindungi Guard

Method	Alamat (Endpoint)	Apakah Perlu Kunci Akses?
GET	/wisata	Tidak — siapa saja bisa mengambil data.
GET	/wisata/:id	Tidak — siapa saja bisa mengambil data berdasarkan ID.
POST	/wisata	Ya — hanya bisa dilakukan jika membawa kunci akses yang benar.
PATCH	/wisata/:id	Ya — hanya bisa dilakukan jika membawa kunci akses yang benar.
DELETE	/wisata/:id	Ya — hanya bisa dilakukan jika membawa kunci akses yang benar.

9. Daftar Endpoint API (wisata.controller.ts)

Controller adalah bagian yang mengatur alamat-alamat (endpoint) API yang bisa diakses oleh frontend. Semua endpoint wisata dimulai dari alamat dasar /wisata.

Method	Alamat	Kunci Akses?	Hasil jika Berhasil	Fungsi
GET	/wisata	Tidak	200 + Daftar semua data	Mengambil seluruh data transaksi wisata yang ada di database.
GET	/wisata/:id	Tidak	200 + Satu data atau 404 jika tidak ada	Mengambil satu data berdasarkan ID yang diberikan.
POST	/wisata	Ya	201 + Data baru yang tersimpan	Menambahkan data transaksi wisata baru ke database.
PATCH	/wisata/:id	Ya	200 + Data yang sudah diperbarui atau 404 jika tidak ada	Mengubah sebagian data berdasarkan ID.
DELETE	/wisata/:id	Ya	204 (Sukses tanpa isi) atau 404 jika tidak ada	Menghapus data berdasarkan ID secara permanen.

Catatan teknis: Parameter :id pada endpoint selalu diproses menggunakan ParseIntPipe, yang memastikan nilainya adalah angka bulat. Jika bukan angka, request akan otomatis ditolak.

10. Logika Utama Aplikasi (wisata.service.ts)

Service adalah tempat semua logika bisnis dijalankan. Controller hanya menerima request dan meneruskannya ke service, sedangkan service yang benar-benar mengerjakan tugasnya, mulai dari mengambil data, menghitung nilai, hingga menyimpan ke database.

10.1 Mengambil Semua Data (findAll)

Mengambil seluruh data yang ada di tabel wisata, kemudian mengirimkannya kembali ke frontend dalam bentuk array JSON.

```
return await this.wisataRepo.find();
```

10.2 Mencari Data Berdasarkan ID (findOne)

Mencari satu baris data di database berdasarkan ID yang diberikan. Jika ID tidak ditemukan, server otomatis mengirimkan error 404 dengan pesan yang informatif, sehingga frontend tahu persis apa yang salah.

```
if (!data) throw new NotFoundException(`ID ${id} tidak ada di database`);
```

10.3 Menambah Data Baru (create)

Menyimpan data kunjungan wisata baru ke database dengan langkah-langkah berikut:

1. Menghitung total secara otomatis: total = Jumlah x Harga. Frontend tidak perlu mengirim nilai total, karena dihitung sendiri oleh server.
2. Membuat objek data baru yang siap disimpan.
3. Menyimpan objek tersebut ke database menggunakan TypeORM.

10.4 Mengubah Data (update)

Memperbarui data yang sudah ada dengan mekanisme partial update, artinya frontend hanya perlu mengirim field yang ingin diubah. Field yang tidak dikirim akan tetap menggunakan nilai lama.

Caranya menggunakan operator ?? (nullish coalescing) dalam JavaScript/TypeScript:

```
const namaBaru = dto.Nama ?? dataLama.nama;
```

Artinya: gunakan dto.Nama jika ada, jika tidak, gunakan dataLama.nama. Nilai total juga akan dihitung ulang secara otomatis setelah update.

10.5 Menghapus Data (remove)

Menghapus data secara permanen dari database. Sebelum menghapus, fungsi findOne() dipanggil terlebih dahulu untuk memastikan data dengan ID tersebut benar-benar ada. Jika tidak ada, proses dihentikan dan error 404 dikirimkan.

11. Alur Kerja Sistem (Request Flow)

Berikut adalah gambaran perjalanan sebuah request dari frontend hingga data kembali dikirimkan:

Langkah	Siapa yang Bertugas	Apa yang Terjadi
1	Frontend (Next.js)	Mengirim HTTP request ke <code>http://localhost:3000/wisata</code>
2	main.ts (ValidationPipe)	Server menerima request. Format data langsung dicek apakah sesuai aturan DTO.
3	WisataGuard	Khusus untuk POST, PATCH, DELETE: header Authorization diperiksa. Jika tidak valid, request ditolak di sini.
4	WisataController	Request diterima dan diteruskan ke method service yang sesuai berdasarkan method HTTP dan alamat endpoint.
5	WisataService	Logika bisnis dijalankan: hitung total, validasi keberadaan data, siapkan data untuk disimpan.
6	TypeORM Repository	Menerjemahkan operasi ke dalam perintah SQL dan mengirimkannya ke database MySQL.
7	Database MySQL	Menjalankan query SQL dan mengembalikan hasilnya.
8	Response ke Frontend	Data hasil operasi dikirim kembali ke frontend dalam format JSON.

12. Pengujian Otomatis (Unit Test)

Project ini menyertakan file-file pengujian yang ditulis menggunakan framework Jest (bawaan NestJS). Pengujian ini berfungsi untuk memastikan setiap komponen utama berjalan tanpa error saat diinisialisasi.

File Pengujian	Komponen yang Diuji	Yang Diperiksa
wisata.controller.spec.ts	WisataController	Memastikan controller berhasil dibuat dan tidak error saat dijalankan.
wisata.service.spec.ts	WisataService	Memastikan service berhasil dibuat dan tidak error saat dijalankan.
wisata.guard.spec.ts	WisataGuard	Memastikan guard keamanan berhasil dibuat dan siap digunakan.
app.controller.spec.ts	AppController	Memastikan endpoint utama "/" mengembalikan teks "Hello World!" dengan benar.

Untuk menjalankan semua pengujian sekaligus, gunakan perintah berikut di terminal:

```
npm run test
```

13. Cara Menjalankan Backend

Yang Perlu Disiapkan Terlebih Dahulu

- **Node.js** versi 18 atau lebih baru sudah terinstal.
- **XAMPP** sudah terinstal dan layanan MySQL sedang berjalan.
- Database **puncak_mas_db** sudah dibuat di phpMyAdmin.

Langkah-Langkah Menjalankan Server

4. Buka terminal dan masuk ke folder backend project.
5. Jalankan perintah "npm install" untuk mengunduh semua dependensi yang diperlukan.
6. Pastikan XAMPP sudah aktif dan MySQL sudah berjalan.
7. Jalankan server dengan perintah "npm run start:dev" untuk mode pengembangan (dengan auto-reload).
8. Jika berhasil, akan muncul pesan di terminal: Server sudah jalan di: http://localhost:3000